

# Data Structure

## 06. 해시 테이블

6주차: Hash Table

## ? 오늘의 핵심 질문

### ? 질문 1

딕셔너리 안에 데이터가 1억개 있어도 원하는 값을 빠르게 찾아낼 수 있는 방법은?

### ? 질문 2

데이터를 찾을 때 순서대로 찾는 방법 말고 어떤 방법이 있을까?

### ? 질문 3

계산결과가 중복되면 어떻게 해결할까?

# 목차 INDEX

## 01 해시 테이블

- Hash Table
- Key & Value
- Hash Function
- Index
- 탐색 과정
- 시간복잡도

## 02 해시 충돌

- Collision
- Chaining
- Open Addressing
- Load Factor
- Rehashing

## 03 딕셔너리

- Dictionary
- Hashable
- get()
- Set

01

# 해시 테이블

Hash Table



## 배열에서 인덱스로 값을 찾는 방법

### 인덱스를 알고 있는 경우

numbers[3] 로 접근 시

메모리 주소를 바로 계산하여 즉시 접근할 수 있음

시간복잡도:  $O(1)$

```
numbers = [10, 20, 30, 40, 50] print(numbers[3]) # 바로 40 출력
```

### 값만 알고 위치를 모르는 경우

40 in numbers 로 검색 시

0번 인덱스부터 하나씩 순서대로 비교해야 함

시간복잡도:  $O(N)$

```
for x in numbers: if x == 40: return True  
# 40이 있는지 찾으려면?
```

## 인덱스 말고 Key를 쓰면 어떨까?

Key

위치를 스스로 알려주는 열쇠

찾고 싶은 값(Key)으로 위치를 직접 계산하자

기존 방식: 데이터가 들어간 자리(index)를 외우거나, 처음부터 탐색함  $O(N)$

새로운 **아이디어**: 데이터의 이름(Key)을 수학적 연산식에 넣어서,  
그 데이터가 저장될 배열의 Index를 즉시 도출

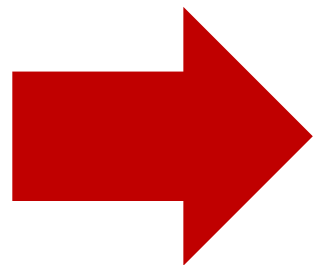
→ Hash Table의 출발점

## 리스트에서 데이터 검색하기

```
people = ["민수", "철수", "영희", "준호", "지수",  
          "현우", "서연", "도윤", "하은", "성민", ...]
```

위의 리스트에서 "성민"을 찾는 상황

1. "민수"와 "성민" 을 비교 → False
2. "철수"와 "성민"을 비교 → False
- ⋮
10. "성민"과 "성민"을 비교 → True



**시간복잡도  $O(N)$**

**별로 안좋은 방법인듯**

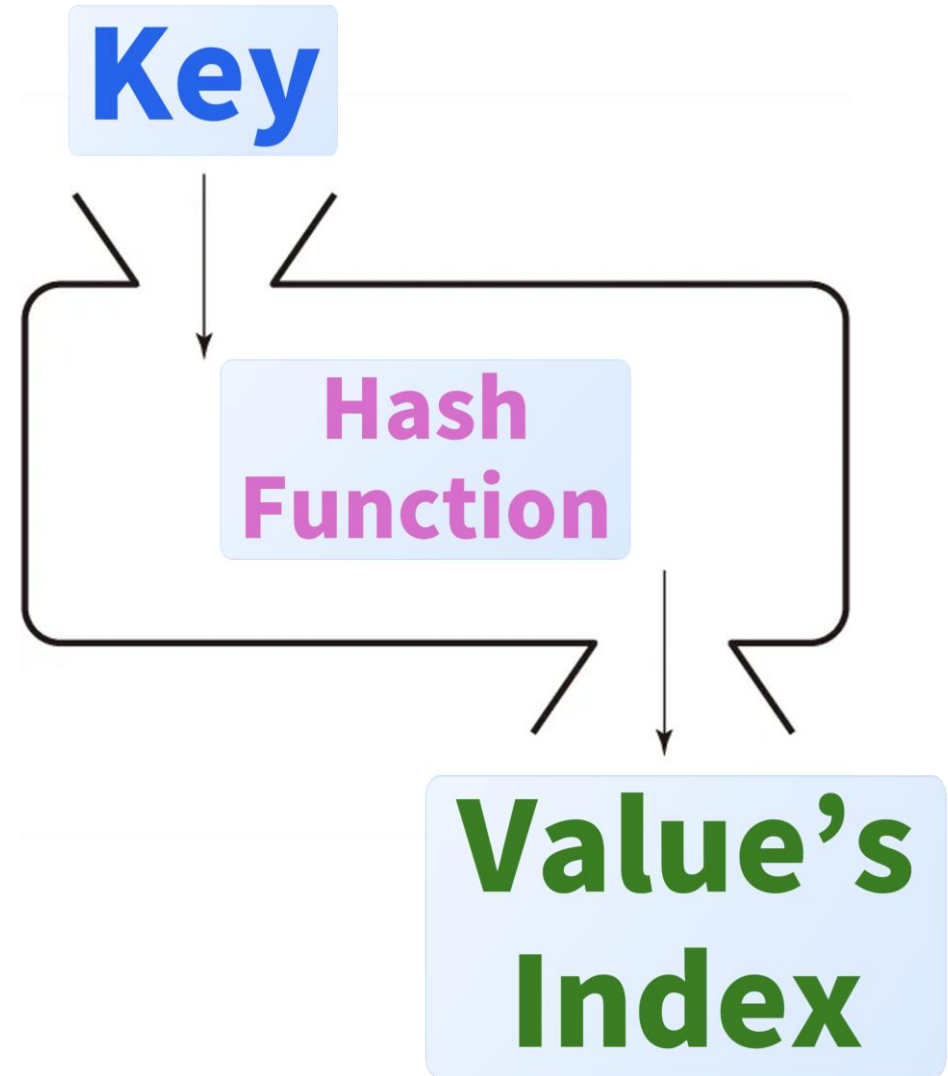
## 해시의 작동 방식

### Key (키)

- 데이터를 찾기 위한 식별자
- 중복될 수 없는 고유한 값
- Ex) 주민등록번호, 학번, 단어, 사용자 ID

### Value (값)

- Key와 연결된 실제 저장 데이터
- 중복이 가능하며 모든 형태 가능
- Ex) 이름, 상세 정보, 단어의 뜻, 회원 객체



## 해시 함수의 조건



### 변환 역할

임의의 길이의 데이터(Key)를 입력받아  
고정된 길이의 정수(Hash Value)로  
변환해야 함



### 결정론적 성질

동일한 Key를 입력하면 언제나  
예외 없이 동일한 Hash Value가  
계산되어 나와야 함



### 초고속 연산

Key에서 Hash Value를 계산하는 과정  
은  $O(1)$ 에 가까울 정도로  
극도로 빨라야 함

## 해시값으로 인덱스 계산하기

- 해시 함수는 어떻게 설계하냐에 따라 해시값이 천차만별임
- 그런데 계산된 해시값이 배열의 길이를 넘어서는 매우 큰 값일 수 있음
- 그래서 이 해시값을 배열 크기에 맞게 줄여야 하는데, 우리는 % 연산자를 이용할거임



모듈로(Modulo, %) 연산 활용  
(나머지 연산자임)

$(\text{index} + 1) \% \text{capacity}$

## 해시값으로 인덱스 계산하기



### 1. Key 입력

탐색하고자 하는 문자열/객체  
(예: "apple")



### 2. Hash Function

Key를 고유 정수로 변환  
(예: 530)



### 3. Index 연산

Hash Value % 크기  
(예:  $530 \% 10 = 0$ )



### 4. Bucket 접근

Index 0번 칸  
(칸 == Bucket 이라고도 부름)  
의 Value 저장/조회

### 2단계

hash("apple") 연산  
→ 530 도출

### 4단계

Bucket[0]의 사과 데이터 반환

### 1단계

Key("apple") 수신

### 3단계

$530 \% 10 \rightarrow$  Index 0 도출

## 해시값으로 인덱스 계산하기

| 연산 (Operation) | 일반 배열 / 순차 탐색    | 해시 테이블 (평균) | 해시 테이블 (최악) |
|----------------|------------------|-------------|-------------|
| Search (탐색)    | $O(N)$           | $O(1)$      | $O(N)$      |
| Insert (삽입)    | $O(1)$ 또는 $O(N)$ | $O(1)$      | $O(N)$      |
| Delete (삭제)    | $O(N)$           | $O(1)$      | $O(N)$      |

해시 테이블의 시간복잡도는  
반드시 "평균"을 붙여야 함

최악의 경우  $O(N)$ 이 되기 때문임



02

# 해시 충돌

Hash Collision

# Hash Collision



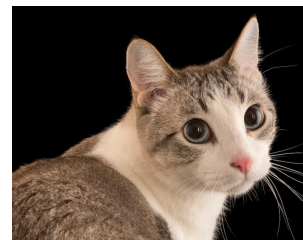
Key: "cat"  
Value: cat.png  
Hash Value: 357



Key: "bird"  
Value: bird.png  
Hash Value: 197

# Hash Collision

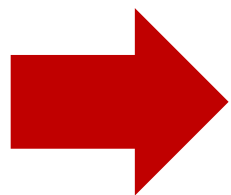
- 이 데이터들을 저장할 Array의 길이가 10인 상태임
- 그럼 cat.png를 저장하려면  $357\%10 = 7$ 이니까 7번 index 자리 (7번 bucket)에 저장하면 될듯
- bird.png도 같은 방법으로 해보자
- bird.png는  $197\%10 = 7$  이니까 7번 index에 저장하면 될듯
- 근데 7번에 이미 cat.png 있는데?



Key: "cat"  
Value: cat.png  
Hash Value: 357



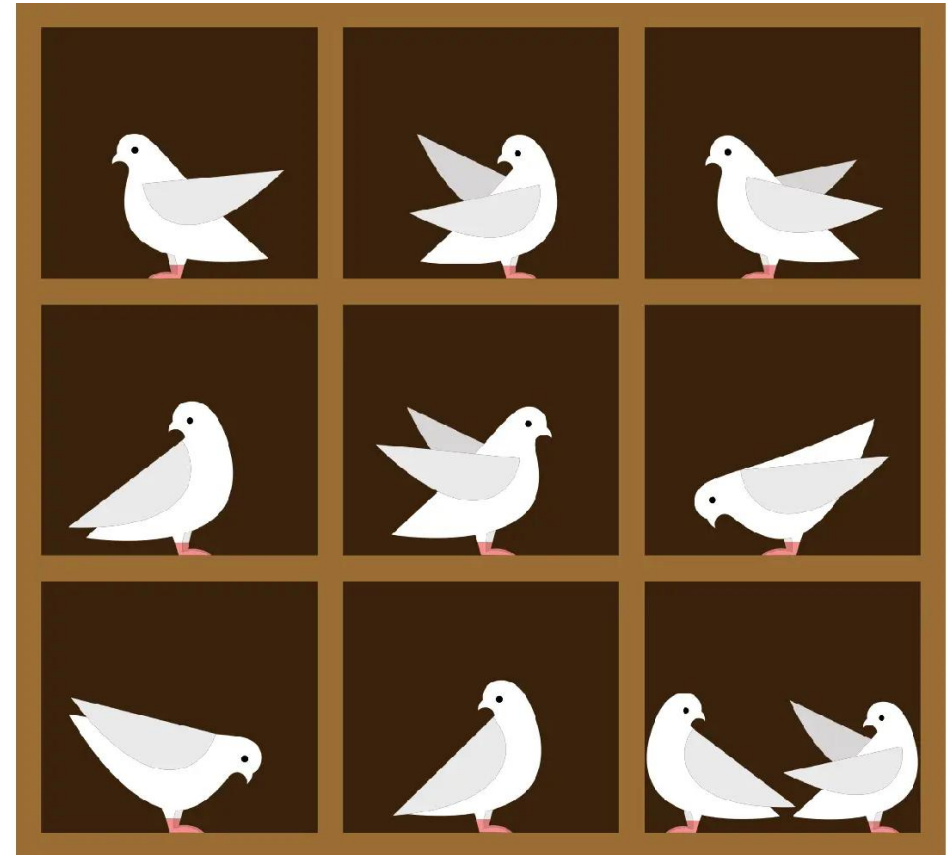
Key: "bird"  
Value: bird.png  
Hash Value: 197



# Hash Collision

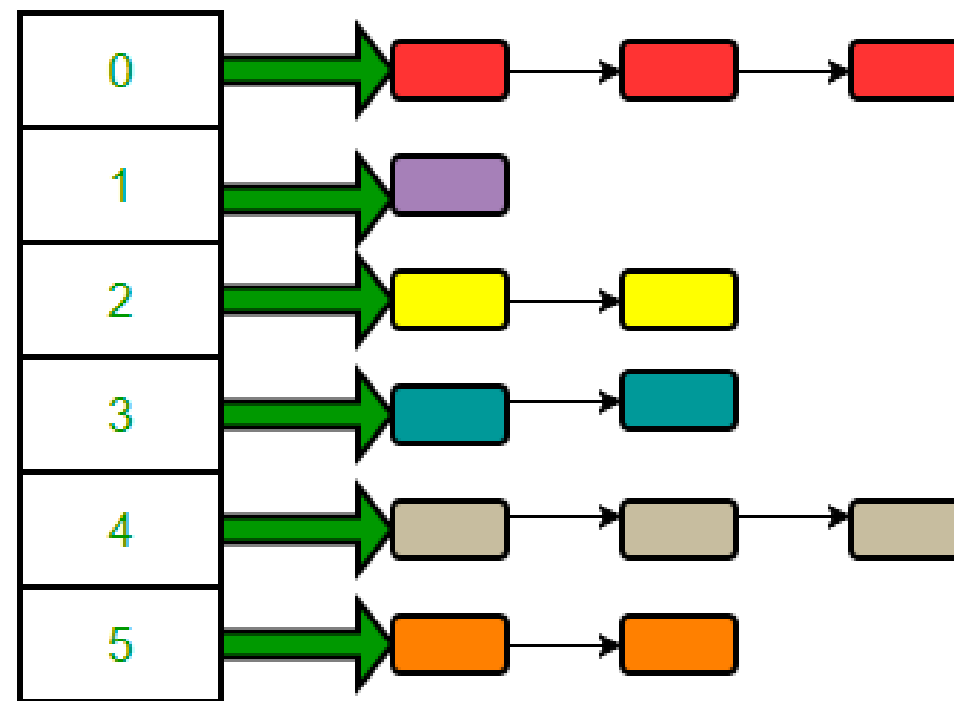
### 비둘기집 원리

- 비둘기가 10마리 있는데 비둘기집이 9개라면 적어도 하나의 집에는 2마리 이상 들어가야 한다는 원리임 (사실 당연한 말임)
- 배열에 들어갈 수 있는 데이터의 개수(비둘기)는 Array의 총 Bucket 수보다 압도적으로 많음
- 반면 해시 테이블로 넣을 수 있는 Bucket의 수(비둘기집)는 유한함 (메모리의 한계 때문)
- 따라서 해시 충돌은 당연히 발생하는 현상이므로 잘 해결하는 것이 핵심임



### 충돌 해결법 1: Chaining

- 각 Bucket (각 Index가 가리키는 배열의 각 칸)들을 Linked List의 Head로 만듦
- 만약 같은 Bucket에 데이터가 또 들어오면 Linked List의 노드를 계속 연결함
- 검색할때는 Bucket을 한 번에 찾은 뒤, 그곳의 Linked List는 하나씩 비교함
- 만약 모든 데이터가 하나의 Bucket에 쏠렸다면 (최악의 경우) 해시가 아니라 그냥 긴 Linked List가 되어 시간복잡도가  $O(N)$ 으로 상승함



# 충돌 해결법 2: Open Addressing

- 충돌이 발생하면 해시 테이블 Array 내부의 다른 비어있는 Bucket을 조사함
- Chaining 대비 Linked List의 노드 등을 사용하지 않으므로 메모리를 좀 절약할 수 있음
- 비어있는 Bucket을 조사하는 방법은 여러 알고리즘이 있음
- 그 중 가장 대표적인게 Linear Probing 방법임

# Linear Probing



쉽게 말해서 계속 옆방 보면서  
비어있는 곳 들어가는 알고리즘임

# Load Factor

$$\alpha = n / k$$

해시 테이블이 얼마나 채워졌는가?

- n: 저장된 데이터(Key)의 총 개수
- k: 해시 테이블의 전체 방(Bucket) 개수

Ex) 방이 10개인데 데이터가 3개 들어가 있다면?

$$\alpha = 3 / 10 = 0.3 \text{ (적재율 30\%)}$$



# Load Factor

Load Factor가 낮을 때 ( $\alpha = 0.2$ )

- 주차장에 차가 20%만 차있음
- 들어가자마자 빈 공간 발견 쉬움
- 충돌(Collision) 발생 확률 매우 낮음

→ 탐색 속도  $O(1)$  유지

Load Factor가 높을 때 ( $\alpha = 0.9$ )

- 주차장에 차가 90% 들어차있음
- 빈 주차 자리를 찾느라 한참 빙빙 돌
- 충돌 빈번, Linear Probe 연속 발생

→ 탐색 속도  $O(N)$ 으로 급락

# Rehashing

### 1. 임계치(Threshold) 도달

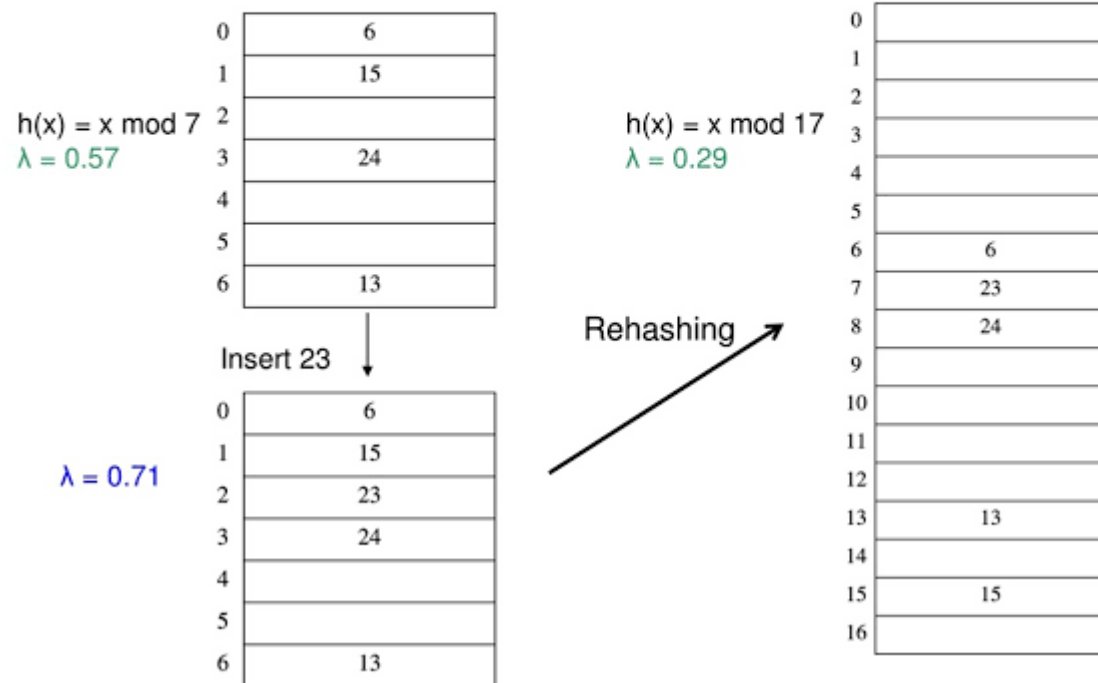
Load Factor가 약 0.6 ~ 0.75 에 도달하면,  
성능 저하를 막기 위해 테이블 크기를 확장

### 2. 테이블 2배 확장 & Rehash

- 1) 기존 크기의 약 2배 되는 새 테이블 생성
- 2) 모든 Key를 새 크기 기준으로 Index 다시 연산
- 3) 새 테이블로 데이터 재배치

해시테이블의 크기가 변경되었으므로  
해시값으로 구하는 인덱스가 완전히 달라짐

## Rehashing Example



03

# 딕셔너리

Dictionary

# Dictionary

```
country_capitals = {  
    'Germany' : 'Berlin',  
    'Canada' : 'Ottawa',  
    'England' : 'London'  
}
```

key value

element 1  
element 2  
element 3

사실 파이썬의 딕셔너리는  
Hash Table을 바탕으로 만들어짐

# Hashable / Unhashable

### Hashable (해시 가능)

- 객체의 생존 기간 동안 해시값이 절대 변하지 않는 객체
  - `__hash__()` 메서드를 구현하고 있음
  - 동등성 비교(==)가 가능함
- dict의 Key나 set의 원소가 될 수 있음

### Unhashable (해시 불가능)

- 값이 변경될 수 있는 객체 (Mutable)
  - 값이 변하면 해시값도 바뀌어야 하므로  
해시 테이블에 안정적으로 저장 불가
- dict의 Key로 사용할 수 없음

# Hashable / Unhashable

| 자료형 (Data Type)  | 가변성 (Mutability) | Hashable 여부                      | dict Key 가능 여부 |
|------------------|------------------|----------------------------------|----------------|
| int, float, bool | Immutable (불변)   | Hashable                         | 가능             |
| str (문자열)        | Immutable (불변)   | Hashable                         | 가능             |
| tuple (튜플)       | Immutable (불변)   | 조건부<br>(내부요소들이 전부 Hashable이면 가능) | 조건부 가능         |
| list, dict, set  | Mutable (가변)     | Unhashable                       | 불가능            |

### dict.get()

```
print(person.get('job'))           # None
print(person.get('job', '없음'))   # 없음
```

1. 'job'이라는 Key를 Hash Function에 넣음
2. Hash Value가 나오면 나머지 연산을 통해 어느 Bucket인지 탐색
3. 해당 Bucket에 'job'이라는 Key가 존재하는지 확인
4. 있으면 반환, 없으니까 '없음' 반환

# Python Set

```
country_capitals = {  
    'Germany' : 'Berlin',  
    'Canada' : 'Ottawa',  
    'England' : 'London'  
}
```

key value

Dictionary

- Key : Value 형태
- Key와 연결된 Value를 얻는 것이 목적
- 중복 허용
- **해시 테이블 기반 구조**

S = { 20, 'Jessa', 35.75 }

Set

- Key 만 존재하는 형태
- Key 가 존재하는지 자체를 확인하는 것이 목적
- 중복 불가
- **해시 테이블 기반 구조**



## 03 딕셔너리 Dictionary

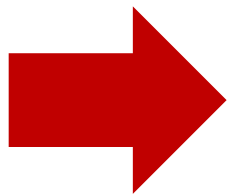
백만개의 데이터에서 999,999 라는 값을 찾는 실험

| 자료형   | 수행한 연산         | 소요 시간                      |
|-------|----------------|----------------------------|
| list  | 999999 in data | 약 6.24 ms (약 6240 $\mu$ s) |
| tuple | 999999 in data | 약 4.68 ms (약 4680 $\mu$ s) |
| dict  | 999999 in data | 약 0.032 $\mu$ s            |
| set   | 999999 in data | 약 0.028 $\mu$ s            |

자료구조 1주차 강의자료  
데이터 타입별 성능 실험



list와 set는 약 **221,429배**의 성능 차이를 보임



리스트와 튜플은 해시 기반이 아니고,  
딕셔너리와 세트는 해시 기반이기 때문